Java on the Command Line

Freshman Level: Your First Programs in a Terminal

Haluk O. Bingol

Faculty of Computer and Information Sciences
Yeditepe University

v2026-10-13

Outline

Installing Java

What you need

1. A **JDK** (Java Development Kit): it contains `javac` (compiler) and `java` (runner)
2. A **terminal**: **Windows** Command Prompt or PowerShell, **macOS** Terminal
3. A **text editor** for plain text: VS Code, Notepad++, Sublime, or the built-in editors (with care)

Which JDK?

Eclipse **Temurin 21** (or 25), from `adoptium.net`. Free, and the installer sets everything up.

JDK, not JRE

A JRE can only run programs; it has no `javac`.

Why the terminal if IDEs exist?

You see exactly what happens; error messages are readable; it works on every computer, server and exam machine.

Installing on Windows

1. Open `adoptium.net`, choose **Temurin 21 (LTS)**, **Windows**, **x64**, **JDK**, `.msi`
2. Run the `.msi`. On *Custom Setup* make sure these are enabled:
   - *Add to PATH*
   - *Set JAVA_HOME variable*
3. Finish, then open a **new** Command Prompt (`Win` key, type `cmd`, Enter)
4. Check: `java -version` and `javac -version`

Or in one command (Windows 10/11): `winget install EclipseAdoptium.Temurin.21.JDK`

'javac' is not recognized ...

- Did you open a **new** window after installing?
- `Add to PATH` missed: *Start*, type *environment variables*, *Edit the system environment variables*, *Environment Variables...*, select `Path`, *Edit*, *New*: `C:\Program Files\Eclipse Adoptium\jdk-21...\bin`
- Check with `where javac`

Installing on macOS

1. Which chip? *Apple menu, About This Mac*: **Apple M1/M2/...** ⇒ `aarch64`; **Intel** ⇒ `x64`
2. Open `adoptium.net`, choose **Temurin 21**, **macOS**, your chip, **JDK**, `.pkg`
3. Run the `.pkg` and follow the steps
4. Open **Terminal** (`Cmd`+`Space`, type *Terminal*, Enter)
5. Check: `java -version` and `javac -version`

With Homebrew: `brew install -cask temurin@21`

"Unable to locate a Java Runtime"

macOS has a `java` command that only says Java is missing. Install the JDK, then open a new Terminal window.

Several JDKs?

`/usr/libexec/java_home -V` lists them. They live in `/Library/Java/JavaVirtualMachines/`.

Checking the installation

Type both commands; each must print a version, not an error:

Terminal codeoutput/Check.txt

```
$ java -version
openjdk version "21.0.12.1" 2026-08-18
OpenJDK Runtime Environment (build 21.0.12.1+1-1-24.04.4-Ubuntu)
OpenJDK 64-Bit Server VM (build 21.0.12.1+1-1-24.04.4-Ubuntu, mixed mode, sharing)
$ javac -version
javac 21.0.12.1
```

Recorded on Linux with Ubuntu's OpenJDK; with Temurin on Windows or macOS the first line says `openjdk version "21..."` and the next lines mention `Temurin`. The version numbers of `java` and `javac` should match.

Living in the Terminal

Moving around in the terminal

Windows (Command Prompt)

```
C:\Users\ayse> cd Desktop
C:\Users\ayse\Desktop> mkdir cse101
C:\Users\ayse\Desktop> cd cse101
C:\Users\ayse\Desktop\cse101> dir
C:\Users\ayse\Desktop\cse101> cd ..
C:\Users\ayse\Desktop> cd
C:\Users\ayse\Desktop
```

macOS (Terminal)

```
ayse@mac ~ % cd Desktop
ayse@mac Desktop % mkdir cse101
ayse@mac Desktop % cd cse101
ayse@mac cse101 % ls
ayse@mac cse101 % cd ..
ayse@mac Desktop % pwd
/Users/ayse/Desktop
```

Want to	Windows	macOS
where am I?	`cd`	`pwd`
list files	`dir`	`ls`
go into / up	`cd name` / `cd ..`	`cd name` / `cd ..`
show a file	`type Hello.java`	`cat Hello.java`

Time savers (both): `Tab` completes names; ↑ repeats earlier commands; drag a folder onto the window to paste its path.

Writing the file: plain text, right name

Windows Notepad

- *Save as type*: **All files (*.*)**, name `Hello.java`; otherwise you get `Hello.java.txt`
- Turn on *File name extensions* in File Explorer (*View*) to see the real name
- Encoding: **UTF-8** (the default in current Windows)

macOS TextEdit

- *Format, Make Plain Text* (`Shift+Cmd+T`) before typing; rich text breaks `javac`
- Turn off smart quotes (*Edit, Substitutions*): Java needs straight `"`, not curly quotes
- When saving, untick *If no extension is provided, use .txt*

Easier: a code editor

VS Code, Notepad++ (Windows) or Sublime Text: plain text by default, colours the code, has a built-in terminal.

Your First Program

Compile and run

codeoutput/First/Hello.java — Java

```java
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

`javac Hello.java` (with `.java`) creates `Hello.class`; `java Hello` (no extension) runs it. No output from `javac` means **success**.

Compile and run

codeoutput/First/Hello.java — Java

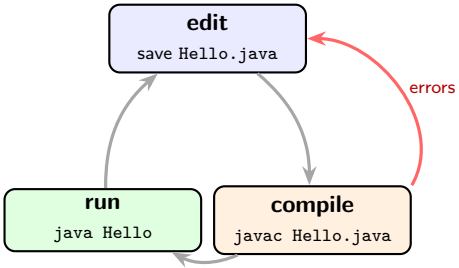
```java
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

Terminal — codeoutput/First.txt

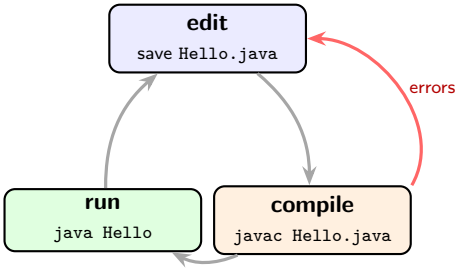
```
$ ls
Hello.java
$ javac Hello.java
$ ls
Hello.class
Hello.java
$ java Hello
Hello, World!
```

`javac Hello.java` (with `.java`) creates `Hello.class`; `java Hello` (no extension) runs it. No output from `javac` means **success**.

The edit–compile–run cycle



The edit–compile–run cycle



Terminal codeoutput/Recompile.txt

```
$ javac Hello.java
$ java Hello
Hello, World!
$ cat Hello.java
public class Hello {
    public static void main(String[] args) {
        System.out.println("Merhaba, Dünya!");
    }
}
$ java Hello
Hello, World!
$ javac Hello.java
$ java Hello
Merhaba, Dünya!
```

Between the first and second `java Hello` the file was changed in the editor and saved, but **not compiled**: `java` still ran the old `Hello.class`.

Reading input with Scanner

codeoutput/Greet/Greet.java — Java

```java
import java.util.Scanner;

public class Greet {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        System.out.print("Your name: ");
        String name = in.nextLine();
        System.out.print("Your age: ");
        int age = in.nextInt();
        System.out.println();
        System.out.println("Hi " + name + ", next year you will be "
                + (age + 1) + ".");
    }
}
```

When **you** run it, you type the answers after the prompts. Here they came from `printf ... |` and from a file (`< answers.txt`), so they are not shown on the screen.

Reading input with Scanner

codeoutput/Greet/Greet.java — Java

```java
import java.util.Scanner;

public class Greet {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        System.out.print("Your name: ");
        String name = in.nextLine();
        System.out.print("Your age: ");
        int age = in.nextInt();
        System.out.println();
        System.out.println("Hi " + name + ", next year you will be "
                + (age + 1) + ".");
    }
}
```

Terminal — codeoutput/Greet.txt

```
$ javac Greet.java
$ printf 'Ayse\n19\n' | java Greet
Your name: Your age:
Hi Ayse, next year you will be 20.
$ java Greet < answers.txt
Your name: Your age:
Hi Mehmet, next year you will be 21.
```

When **you** run it, you type the answers after the prompts. Here they came from `printf ... |` and from a file (`< answers.txt`), so they are not shown on the screen.

Command-line arguments

codeoutput/Args/Area.java — Java

```java
public class Area {
    public static void main(String[] args) {
        if (args.length != 2) {
            System.out.println("usage: java Area <width> <height>");
            return;
        }
        try {
            double w = Double.parseDouble(args[0]);
            double h = Double.parseDouble(args[1]);
            System.out.println("area = " + (w * h));
        } catch (NumberFormatException e) {
            System.out.println("not a number: " + e.getMessage());
        }
    }
}
```

Arguments are **strings**; `Double.parseDouble` converts them and throws `NumberFormatException` on bad input. Check `args.length` first and catch the exception.

Command-line arguments

codeoutput/Args/Area.java — Java

```java
public class Area {
    public static void main(String[] args) {
        if (args.length != 2) {
            System.out.println("usage: java Area <width> <height>");
            return;
        }
        try {
            double w = Double.parseDouble(args[0]);
            double h = Double.parseDouble(args[1]);
            System.out.println("area = " + (w * h));
        } catch (NumberFormatException e) {
            System.out.println("not a number: " + e.getMessage());
        }
    }
}
```

Terminal — codeoutput/Args.txt

```
$ javac Area.java
$ java Area 3 4
area = 12.0
$ java Area 2.5 10
area = 25.0
$ java Area 3
usage: java Area <width> <height>
$ java Area three 4
not a number: For input string: "three"
```

Arguments are **strings**; `Double.parseDouble` converts them and throws `NumberFormatException` on bad input. Check `args.length` first and catch the exception.

First-Week Errors

Error 0: the command is not found

Windows (Command Prompt)

```
C:\Users\ayse\cse101> javac Hello.java
'javac' is not recognized as an internal or
external command, operable program or batch file.
```

macOS (Terminal)

```
ayse@mac cse101 % javac Hello.java
zsh: command not found: javac
```

- Not a Java error: the **terminal** cannot find the program
- Fix: install the JDK, open a **new** terminal window, check `PATH` (Windows: `where javac`, macOS: `which javac`)
- `java` works but `javac` does not? A JRE or an old Java is first on the `PATH`

Compile errors: file name and class name

Terminal codeoutput/WrongFile.txt

```
$ ls
Greeting.java
$ javac Greeting.java
Greeting.java:1: error: class Hello is public,
     should be declared in a file named Hello.
     java
public class Hello {
       ^
1 error
```

A `public class Hello` must be in `Hello.java`, with the same capital letters.

Terminal codeoutput/NoFile.txt

```
$ ls
Hello.java
$ javac Helo.java
error: file not found: Helo.java
Usage: javac <options> <source files>
use --help for a list of possible options
```

A typo in the file name, or the file is in another folder: check with `ls` (**Windows**: `dir`).

Compile errors: typos

codeoutput/Typos/Hello.java **Java**

```java
public class Hello {
    public static void main(String[] args) {
        system.out.println("Hello, World!");
        System.out.printn("Bye");
    }
}
```

Java is **case-sensitive**: `system` is not `System` ("package system does not exist"). **cannot find symbol** means "I don't know this name": here a typo, `printn`. Each error gives file, line and a ^.

Compile errors: typos

codeoutput/Typos/Hello.java — Java

```java
public class Hello {
    public static void main(String[] args) {
        system.out.println("Hello, World!");
        System.out.printn("Bye");
    }
}
```

Terminal — codeoutput/Typos.txt

```
$ javac Hello.java
Hello.java:3: error: package system does not exist
        system.out.println("Hello, World!");
               ^
Hello.java:4: error: cannot find symbol
        System.out.printn("Bye");
                   ^
  symbol:   method printn(String)
  location: variable out of type PrintStream
2 errors
```

Java is **case-sensitive**: `system` is not `System` ("package system does not exist"). **cannot find symbol** means "I don't know this name": here a typo, `printn`. Each error gives file, line and a ^.

Run errors: what you give to `java`

Terminal codeoutput/RunErrors.txt

```
$ javac Hello.java
$ java Hello.class
Error: Could not find or load main class Hello.class
Caused by: java.lang.ClassNotFoundException: Hello.class
$ java hello
Error: Could not find or load main class hello
Caused by: java.lang.ClassNotFoundException: hello
$ java Helo
Error: Could not find or load main class Helo
Caused by: java.lang.ClassNotFoundException: Helo
```

`java` wants the **class name**: no `.class`, same capitals, no typos. On Windows and macOS (case-insensitive disks) `java hello` may instead report `wrong name: Hello`; the fix is the same.

Run errors: no `main` method

codeoutput/NoMain/Tools.java — Java

```java
public class Tools {
    static int square(int x) {
        return x * x;
    }
}
```

A class can compile without `main`, but only a class with `public static void main(String[] args)` can be started with `java`.

Run errors: no `main` method

codeoutput/NoMain/Tools.java — Java

```java
public class Tools {
    static int square(int x) {
        return x * x;
    }
}
```

Terminal — codeoutput/NoMain.txt

```
$ javac Tools.java
$ java Tools
Error: Main method not found in class Tools, please define the main method as:
   public static void main(String[] args)
or a JavaFX application class must extend javafx.application.Application
```

A class can compile without `main`, but only a class with `public static void main(String[] args)` can be started with `java`.

Running a Program You Were Given

Running a JAR file

Your instructor gives you `quiz.jar`. You do not need its source code or `javac`:

Terminal codeoutput/RunJar.txt

```
$ ls
quiz.jar
$ java -jar quiz.jar
Question 1: 1 x 7 = 7
Question 2: 2 x 7 = 14
Question 3: 3 x 7 = 21
$ java -jar quiz.jar 5
Question 1: 1 x 7 = 7
Question 2: 2 x 7 = 14
Question 3: 3 x 7 = 21
Question 4: 4 x 7 = 28
Question 5: 5 x 7 = 35
```

A **JAR** packs compiled classes into one file. Double-clicking it may start it without a terminal window, so you would not see what it prints: run console programs with `java -jar` in a terminal (Windows and macOS alike).

Summary

Cheat sheet

Task	Windows (Command Prompt)	macOS (Terminal)
check Java	`java -version`	`java -version`
go to your folder	`cd Desktop\cse101`	`cd Desktop/cse101`
list files	`dir`	`ls`
compile	`javac Hello.java`	`javac Hello.java`
run	`java Hello`	`java Hello`
run with arguments	`java Area 3 4`	`java Area 3 4`
input from a file	`java Greet < answers.txt`	`java Greet < answers.txt`
output to a file	`java Hello > out.txt`	`java Hello > out.txt`
run a JAR	`java -jar quiz.jar`	`java -jar quiz.jar`
stop a running program	`Ctrl+C`	`Ctrl+C`

Do and do not

Do

- Name the file exactly like the public class
- Compile again after every change
- Read the **first** error message, line number included
- Keep each exercise in its own folder
- Use a plain-text code editor

Do not

- `java Hello.class` (give the class name: `java Hello`)
- `javac Hello` (it needs `.java`)
- Type `system` or `Main` where Java expects `System` or `main`
- Save as `.txt` or rich text
- Work in the Downloads folder

Exercises

1. Install the JDK and show `java -version` and `javac -version` in a terminal.
2. Create a folder `cse101/lab1` from the terminal; write, compile and run `Hello.java` there.
3. Make each of the errors on the "First-Week Errors" slides on purpose and read the message.
4. Write `Rectangle.java` that takes width and height as arguments and prints area and perimeter.
5. Write `Average.java` that reads numbers with `Scanner` until the input ends; test it with `<` and a file.
6. Save the output of a program to `out.txt` with `>` and open it in your editor.
7. Run `quiz.jar` with and without an argument.

Questions?